

Deployment Manual

Deploy SEMS2 on Dokploy with self-hosted Supabase

Deploying SEMS2 to Production

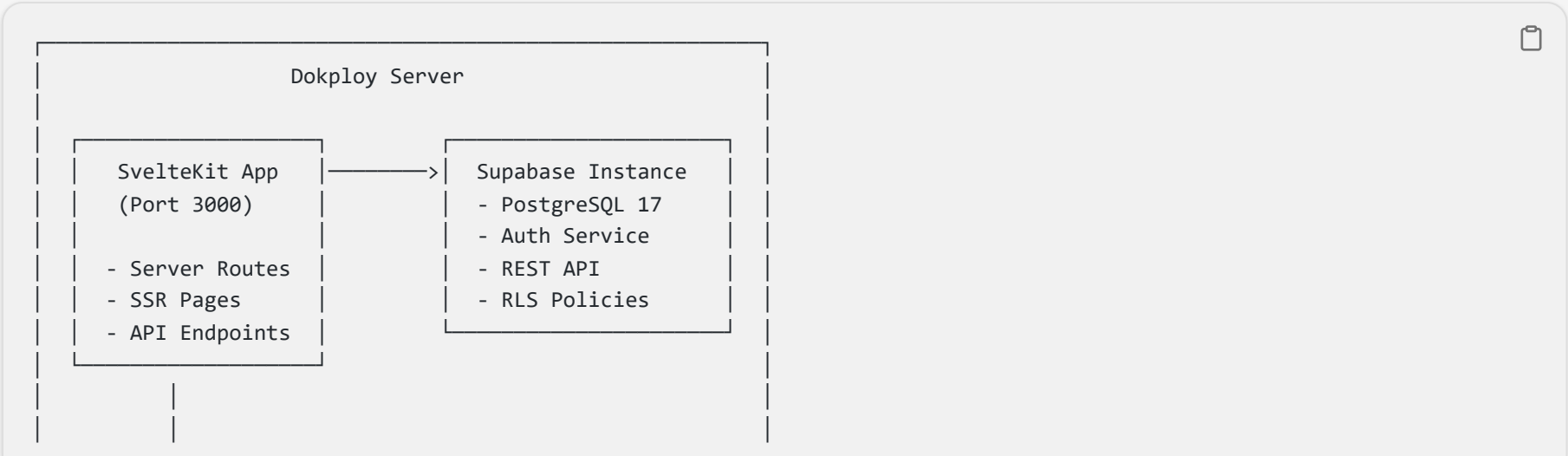
This guide provides comprehensive instructions for deploying the Student Experience Management System (SEMS2) on Dokploy with a self-hosted Supabase instance.

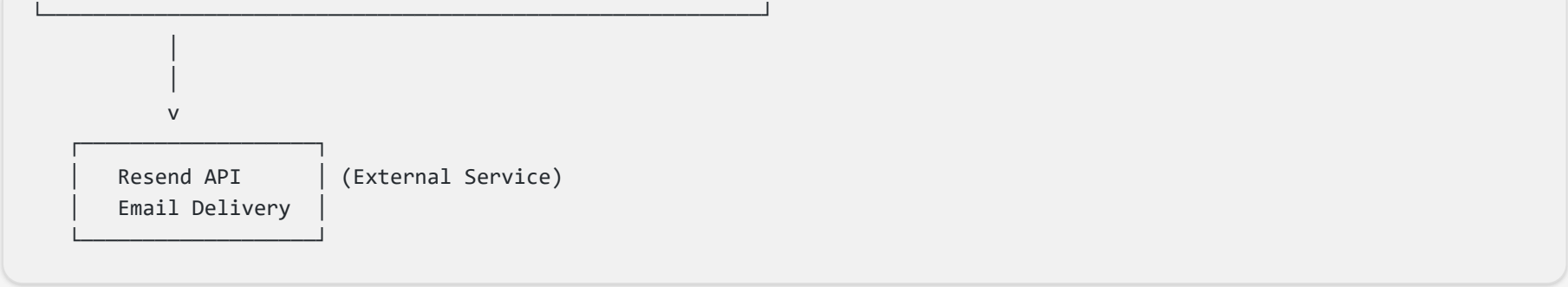
Overview

SEMS2 is a monolithic SvelteKit application that manages events, registrations, and student organizations. The deployment architecture consists of:

- **Frontend & Backend:** SvelteKit 2.16 monolithic application with server-side rendering and API routes
- **Database:** PostgreSQL 17 via self-hosted Supabase
- **Email Service:** Resend for transactional email delivery
- **Authentication:** Supabase Auth with Google OAuth integration
- **Deployment Platform:** Dokploy PaaS (Platform as a Service)

Architecture Overview





Environment Variables Flow

Environment variables are injected at different stages of the deployment lifecycle:

- **Build-time** (`PUBLIC_*`): Embedded into the Docker image during the build process
 - `PUBLIC_SUPABASE_URL`
 - `PUBLIC_SUPABASE_ANON_KEY`
- **Runtime** (Server-only): Injected when the container starts
 - `SUPABASE_SERVICE_ROLE_KEY`
 - `RESEND_API_KEY`
 - `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`

Prerequisites

Before beginning the deployment process, ensure the following requirements are met:

Required Access & Accounts

- **Linux VPS or Dedicated Server:** A server running Ubuntu LTS with Dokploy PaaS installed
- **Dokploy Account:** Administrative access to the Dokploy server dashboard
- **Domain Name:** A registered domain for the SEMS2 deployment (e.g., `sems2.university.edu`)
- **Git Repository:** Access credentials for the SEMS2 repository
- **Resend Account:** API access for transactional email delivery (resend.com)
- **Google Cloud Console:** OAuth 2.0 credentials for authentication (console.cloud.google.com)

Local Development Tools

- **Docker:** Required for running the Supabase CLI
- **Supabase CLI:** For executing database migrations

```
npm install -g supabase
```

- **Node.js 20+:** Required for local build testing (optional)
- **Git:** For repository management

Technical Knowledge

- Basic understanding of Docker containerization
- Familiarity with environment variable configuration
- Basic PostgreSQL/SQL knowledge for database migrations
- Understanding of DNS record management

Deployment Steps

1

Provision a Cloud Server

Deploy an Ubuntu LTS server from a cloud infrastructure provider. The instructions below reference DigitalOcean, which was used during the initial SEMS2 prototype development. However, any cloud provider that supports Ubuntu LTS deployment is suitable.

1. **Create a new server instance:**
 - Operating System: Ubuntu 24.04 LTS
 - Minimum Specifications: 8GB RAM, 4 vCPUs (required for Docker image builds via Dokploy)
 - Security: Configure SSH keys for secure access
 - Initialize and deploy the server instance

2. **Update the system:**

```
sudo apt update && sudo apt full-upgrade -y
```

3. **Implement security hardening:**

Configure essential security measures including:

- UFW (Uncomplicated Firewall) configuration
- SSH key-only authentication
- Root login disabled
- Fail2ban installation (recommended)

For comprehensive security guidance, refer to the [Ubuntu Security Hardening Guide](#).

2 Install Dokploy PaaS

Install Dokploy on the provisioned server by following the official [Dokploy Installation Guide](#). The installation process configures Docker, networking, and the Dokploy dashboard.

3 Deploy Supabase Instance

Deploy a self-hosted Supabase instance using the official Dokploy Supabase template.

1. Create a new Dokploy project:

- Navigate to the Dokploy dashboard
- Create a new project (e.g., "SEMS2 Production")

2. Initialize Supabase service:

- Within the project, select **Create Service** → **Template**
- Filter templates by "Supabase"
- Select the Supabase template (version 1.25.04, Dokploy ≥ 0.22.5)

3. Configure Google OAuth integration:

Modify the `docker-compose.yml` file to enable Google OAuth. Locate the `auth` service and add the following environment variables:

```
auth:
  environment:
    # Enable External Google Auth
    GOTRUE_EXTERNAL_GOOGLE_ENABLED: ${EXTERNAL_GOOGLE_AUTH_ENABLED}
    GOTRUE_EXTERNAL_GOOGLE_CLIENT_ID: ${GOOGLE_CLIENT_ID}
    GOTRUE_EXTERNAL_GOOGLE_SECRET: ${GOOGLE_CLIENT_SECRET}
```



 Reference Configuration

A complete `docker-compose.yml` configuration example from the SEMS2 prototype is available at: `sems2-prototype/dokploy/supabase/docker-compose.yml`

4. **Configure environment variables:**

Navigate to the Supabase service in Dokploy and configure the following environment variables:

Variable	Description	Example Value
<code>POSTGRES_PASSWORD</code>	PostgreSQL admin password	(auto-generated or custom)
<code>SITE_URL</code>	Main application URL	<code>https://sems2.university.edu</code>
<code>GOOGLE_CLIENT_ID</code>	OAuth client ID	(configured in Step 5)
<code>GOOGLE_CLIENT_SECRET</code>	OAuth client secret	(configured in Step 5)
<code>EXTERNAL_GOOGLE_REDIRECT_URI</code>	OAuth callback URL	<code>https://<supabase-domain>/auth/v1/callback</code>

 Known Issue: JWT Key Generation

The Dokploy Supabase template may not correctly generate the following JWT authentication keys:

- `JWT_SECRET` : 32-40 character alphanumeric secret
- `ANON_KEY` : JWT token for anonymous/public access
- `SERVICE_ROLE_KEY` : JWT token for administrative access

Workaround:

1. Visit the [Supabase API Key Generation Guide](#)
2. Generate a 40-character alphanumeric `JWT_SECRET`
3. Generate corresponding `ANON_KEY` and `SERVICE_ROLE_KEY` tokens
4. Update these values in the Supabase service's Environment tab
5. Save the environment configuration
6. Redeploy the service from the General tab

Issue Tracker:[Dokploy/templates#317](#)

5. Reference environment configuration:

Reference Configuration

An example environment configuration from the SEMS2 prototype is available at: `sems2-prototype/dokploy/supabase/.env.example`

6. Configure custom domain:

- Navigate to the Supabase service → **Domains** tab
- Add a custom domain (e.g., `supabase.sems-maintainers.com`)
- Service: `kong`
- Port: `8000`
- Enable HTTPS with Let's Encrypt for automatic SSL certificate provisioning

7. Retrieve connection credentials:

After successful deployment:

- Access the Supabase dashboard (e.g., `https://supabase.sems-maintainers.com`)
- Click **Connect** in the top-left corner
- Select the **App Frameworks** tab
- Record the following values:
 - `PUBLIC_SUPABASE_URL`
 - `PUBLIC_SUPABASE_ANON_KEY`
- These credentials, along with the `SERVICE_ROLE_KEY` generated earlier, are required for subsequent configuration steps

Execute Database Migrations

Apply the SEMS2 database schema to the Supabase instance using the Supabase CLI.

1. **Navigate to the project root** on the local development machine:

```
cd sems2-prototype
```



2. Configure SSL mode for the database connection:

Windows (PowerShell):

```
$env:PGSSLMODE = "disable"
```



macOS/Linux (Bash):

```
export PGSSLMODE=disable
```



3. Execute migrations:

```
supabase db push --db-url "postgres://postgres:<POSTGRES_PASSWORD>@<supabase-host>:5432/postgres"
```



Replace placeholders:

- `<POSTGRES_PASSWORD>`: The PostgreSQL password configured in Step 3
- `<supabase-host>`: The Supabase domain (e.g., `supabase.sems-maintainers.com`)

This command executes all migration files sequentially, creating:

- Core database tables (profiles, events, organizations, registrations)
- Lookup tables (roles, statuses, event states)
- Row Level Security (RLS) policies
- Audit trail triggers and functions

4. Verify migration success:

Access Supabase Studio and confirm:

- **Table Editor**: All tables are present with correct schemas
- **Authentication → Policies**: RLS policies are enabled and configured

💡 **Advanced Migration Management**

Configure Google OAuth

Configure Google OAuth 2.0 credentials for user authentication.

1. Follow the official Supabase guide:

Refer to the [Supabase Google OAuth Documentation](#) for comprehensive setup instructions.

2. Configuration summary:

- Create a new project in Google Cloud Console
- Configure OAuth 2.0 credentials
- Set authorized redirect URIs:
 - `https://<supabase-domain>/auth/v1/callback`
- Obtain the Client ID and Client Secret
- Update the Supabase environment variables (completed in Step 3)

3. Reference configuration:

The image below shows the OAuth configuration used during the SEMS2 prototype development:

Google Cloud

SEMS2

Search (/) for resources, docs, products and more

Q Search

Google Auth Platform / Clients / Client: 84[REDACTED]592643-96g8c[REDACTED]omcnk.apps.googleusercontent.com

Overview

Branding

Audience

Clients

Data access

Verification centre

Settings

Client ID for Web application

Delete

Name *
SEMS2

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorised domains](#).

Authorised JavaScript origins

For use with requests from a browser

URIs 1 *
https://sems2.app

URIs 2 *
http://localhost:5173

+ Add URI

Authorised redirect URIs

For use with requests from a web server

URIs 1 *
http://api.sems2.app/auth/v1/callback

URIs 2 *
http://localhost:5173/auth/v1/callback

URIs 3 *
http://baas.sems2.app/auth/v1/callback

URIs 4 *
http://127.0.0.1:54321/auth/v1/callback

+ Add URI

Note: It may take five minutes to a few hours for settings to take effect

Save Cancel

Additional information

Client ID
84[REDACTED]592643-96[REDACTED]omcnk.apps.googleusercontent.com

Creation date
21 August 2025, 22:28:08 GMT+8

Last used date
27 October 2025 (Note: this data could be delayed by a day or more.)

Inactive OAuth clients are subject to deletion if they are not used for six months. You will be notified of deletion due to inactivity, and can restore clients up to 30 days after deletion. [Learn more](#)

Client secrets

If you are in the process of changing client secrets, you can manually rotate them without downtime. [Learn more](#)

Viewing and downloading client secrets is no longer available. If you have lost the secret below, please add a new one. Remember to store client secrets securely and treat them with extreme care. Secrets should never be checked into code repositories. [Learn more](#)

Client secret
[REDACTED]

Creation date
21 August 2025, 22:28:08 GMT+8

Status
Enabled

+ Add secret

Prototype Reference

The configuration shown includes additional authorized redirect URIs from the development environment. Only the production callback URI is required for the deployment.

6 Configure Resend Email Service

Set up Resend for transactional email delivery, including event notifications, reminders, and certificate generation.

1. Create a Resend account:

Register at resend.com and verify the account.

2. Configure and verify the domain:

- Navigate to **Domains** → **Add Domain**
- Enter the domain name (e.g., `university.edu`)
- Add the provided DNS records to the domain registrar:
 - SPF record
 - DKIM record
 - DMARC record
- Wait for DNS propagation and verification (typically 5-15 minutes)

3. Generate API key:

- Navigate to **API Keys** → **Create API Key**
- Configuration:
 - Name: `SEMS2 Production`
 - Permission Level: **Send Access**
- Copy the generated API key (format: `re_...`)
- Store securely for use in Step 7

7

Deploy SvelteKit Application

Deploy the main SEMS2 application to Dokploy.

1. Create a new application service:

In the Dokploy dashboard:

- Navigate to the SEMS2 project
- Select **Create Service** → **Application**
- Name: `frontend`

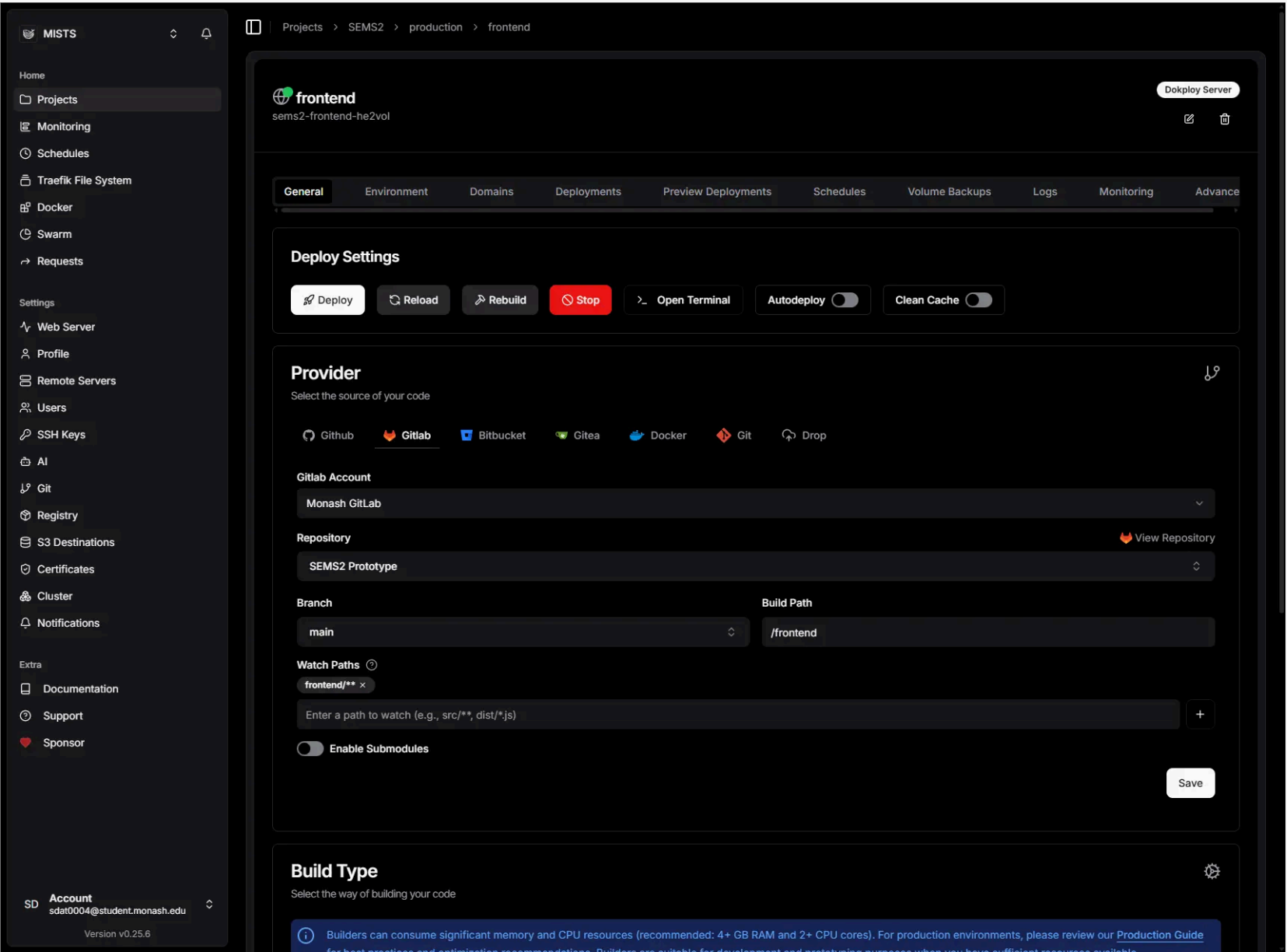
2. Configure Git integration:

- In the Dokploy sidebar, navigate to **Git**
- Select **Add Git Provider**
- Add GitLab and follow the authentication flow

- Return to the project and configure:
 - Provider: GitLab
 - Account: (select authenticated account)
 - Repository: `sems2-prototype`
 - Branch: `main`
 - Build Path: `/frontend`
 - Watch Paths: `/frontend/**`

 Prototype Reference

The configuration shown below demonstrates the setup used during prototype development:



3. Configure build settings:

- Build Type: **Dockerfile**
- Dockerfile Path: `Dockerfile`

4. Configure environment variables:

Navigate to the **Environment** tab:

Runtime Environment Variables (Environment Settings):

Key	Value
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Service role key from Step 3
<code>RESEND_API_KEY</code>	API key from Step 6 (format: <code>re_...</code>)

Build-time Arguments:

Key	Value
<code>PUBLIC_SUPABASE_URL</code>	Supabase URL from Step 3
<code>PUBLIC_SUPABASE_ANON_KEY</code>	Anonymous key from Step 3

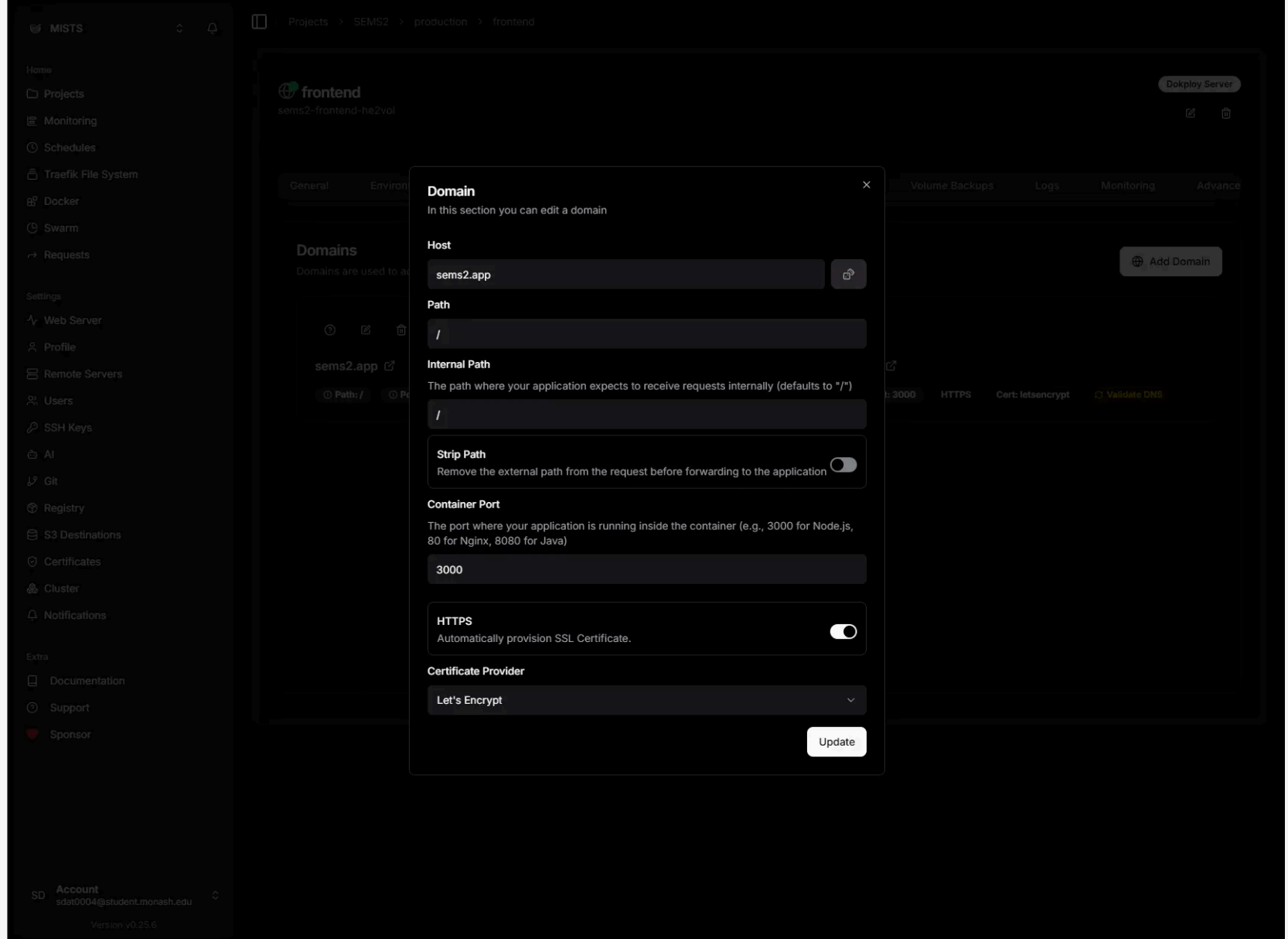
5. Configure custom domain:

Navigate to the **Domains** tab:

- Click **Add Domain**
- Domain: `sems.university.edu`
- Port: `3000`
- Enable HTTPS with Let's Encrypt

 Prototype Reference

Example domain configuration from prototype deployment:



6. Deploy the application:

- Click the **Deploy** button
- Monitor the build logs for errors
- Verify successful deployment

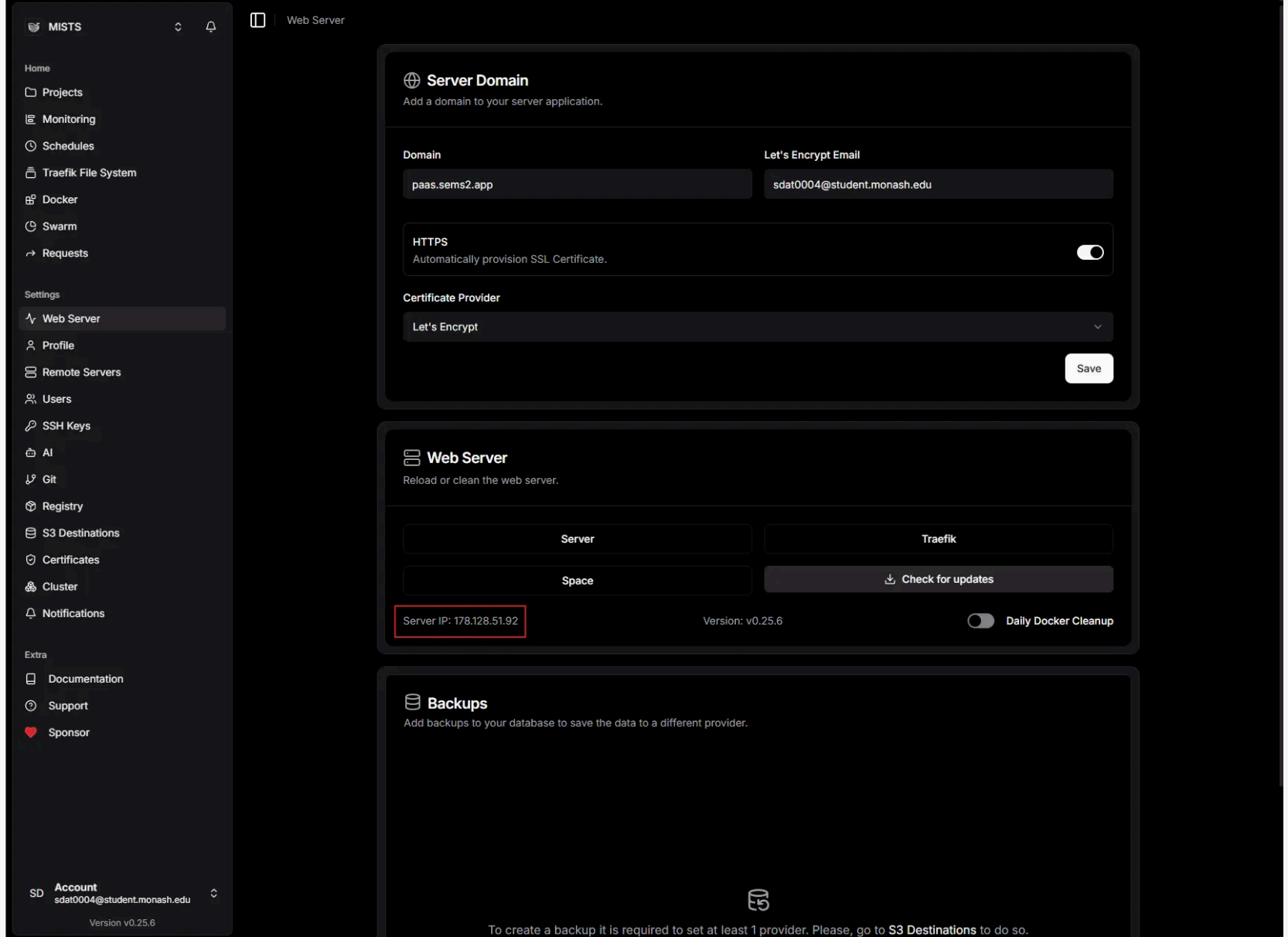
8

Configure DNS Records

Configure DNS records in the domain registrar to route traffic to the Dokploy server.

1. Obtain the server IP address:

Navigate to the Dokploy server dashboard to find the public IP address:



2. Add DNS records:

In the domain registrar's DNS management interface, add A records for each service:

Prototype Reference

Example DNS configuration from prototype deployment:

Sdat0004@student.monash.edu's Account

Go to...Ctrl+KAdd

sems2.appActiveStarFree plan

Overview

RecentsNew

Recordssems2.app > DNSsems2.appDomainsRecordsarchetypediscovery.dev > DNSarchetypediscovery.devDomains

AI Crawl Control

Log Explorer

Analytics & logs

DNSRecordsAnalyticsSettings

Email

SSL/TLS

Security

Access

Speed

Caching

Workers Routes

Rules

Error PagesNew

Network

Traffic

Scrape Shield

Web3

DNS

Records

Configure DNS records and review proxy status of your hostnames.

DNS documentation

DNS management for sems2.app

DNS Setup:FullImport and ExportDashboard Display Settings

Review, add, and edit DNS records. Edits will go into effect once saved.

Search DNS Records

Add filter

Search

Add record

	Type	Name	Content	Proxy status	TTL	Actions
☐	A	baas	178.128.51.92	DNS only	Auto	Edit
☐	A	docs	178.128.51.92	DNS only	Auto	Edit
☐	A	paas	178.128.51.92	DNS only	Auto	Edit
☐	A	registry	178.128.51.92	DNS only	Auto	Edit
☐	A	sems2.app	178.128.51.92	DNS only	Auto	Edit
☐	CNAME	www	sems2.app	DNS only	Auto	Edit
☐	MX	send	feedback-smtp.ap-nort...10	DNS only	1 hr	Edit
☐	TXT	_dmarc	"v=DMARC1; p=reject; sp=...	DNS only	Auto	Edit
☐	TXT	*_domainkey	"v=DKIM1; p=	DNS only	Auto	Edit
☐	TXT	resend_domainkey	"p=MIGfMA0GCsGqGSIlb3D...	DNS only	1 hr	Edit
☐	TXT	sems2.app	"v=spf1 -all"	DNS only	Auto	Edit
☐	TXT	send	"v=spf1 include:amazonses...	DNS only	1 hr	Edit

Cloudflare Nameservers

Every DNS zone on Cloudflare is assigned a set of Cloudflare-branded nameservers.

Type	Value
NS	ines.ns.cloudflare.com
NS	...

3. Verify DNS propagation:

DNS changes may take 5-60 minutes to propagate globally. Use online DNS lookup tools to verify propagation.

9

Deploy Documentation Site

Deploy the Fumadocs documentation as a separate service.

1. Create a new application service:

In the Dokploy dashboard:

- Navigate to the SEMS2 project
- Select Create Service → Application

- Name: docs

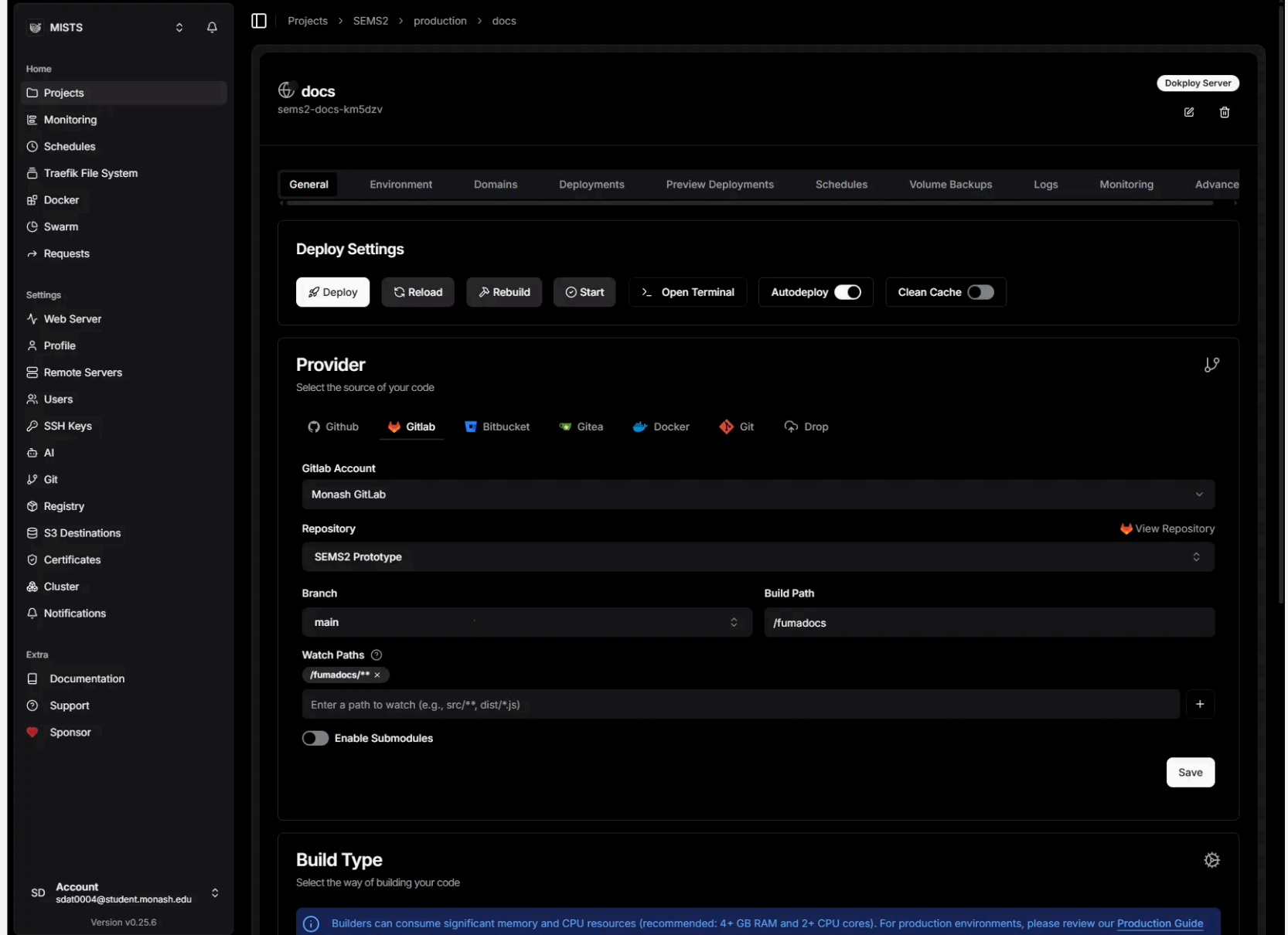
2. Configure Git integration:

- Provider: GitLab
- Account: (authenticated account)
- Repository: sems2-prototype
- Branch: main
- Build Path: /fumadocs
- Watch Paths: /fumadocs/**



Prototype Reference

Example configuration from prototype deployment:



3. Configure build settings:

- Build Type: Dockerfile
- Dockerfile Path: Dockerfile

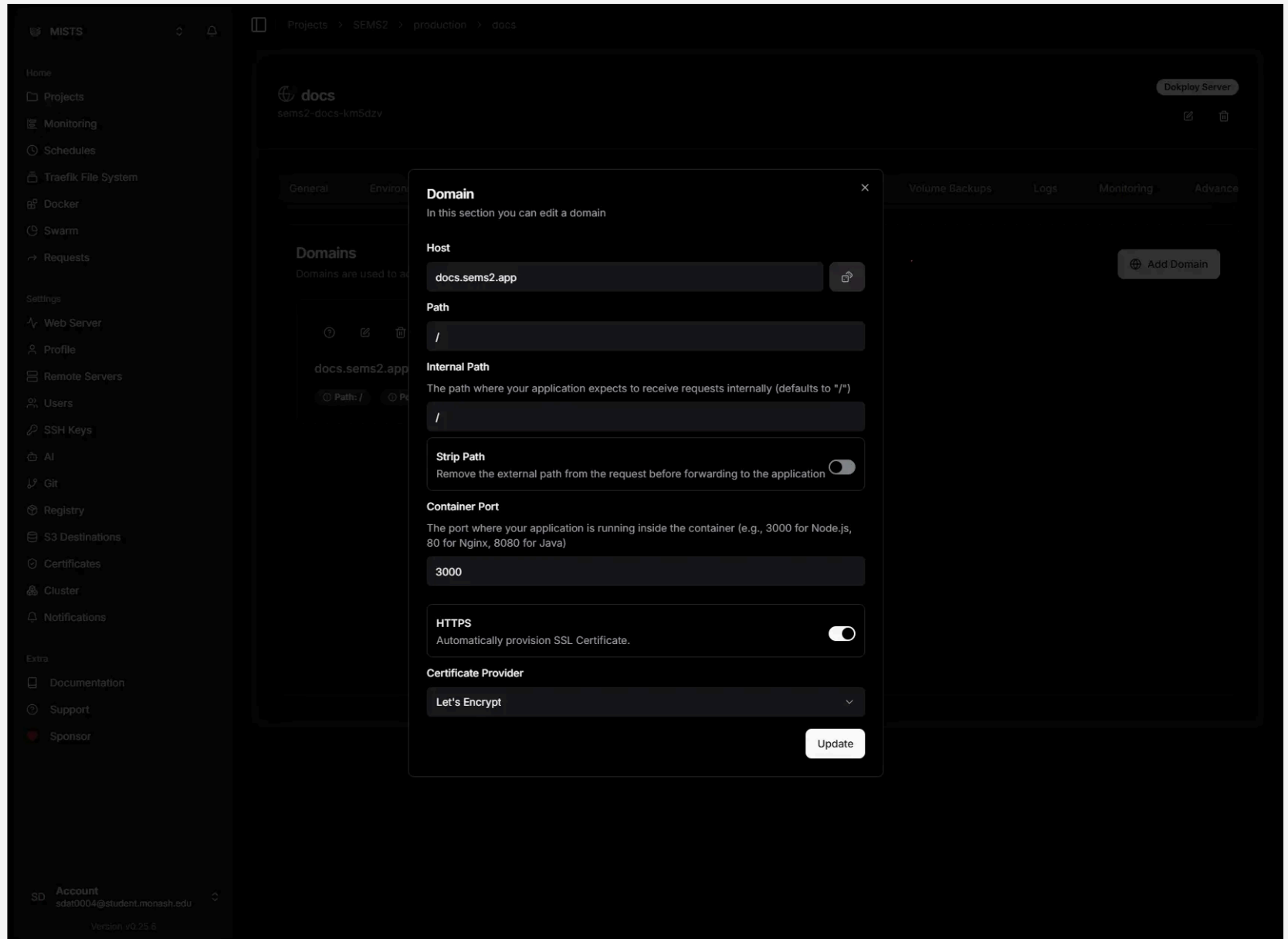
4. Configure custom domain:

Navigate to the Domains tab:

- Click Add Domain
- Domain: docs.sems.university.edu
- Port: 3000
- Enable HTTPS with Let's Encrypt

Prototype Reference

Example domain configuration from prototype deployment:



5. Deploy the application:

- Click the **Deploy** button
- Monitor the build logs for errors
- Verify successful deployment

Post-Deployment Verification

After completing all deployment steps, verify the following:

1. **Application Access:** Confirm all services are accessible via their configured domains
2. **Authentication:** Test Google OAuth login flow
3. **Database Connectivity:** Verify data persistence and retrieval
4. **Email Delivery:** Send test emails to confirm Resend integration
5. **SSL Certificates:** Verify HTTPS is functioning on all domains

Troubleshooting

Common issues and solutions:

- **Build failures:** Check Docker daemon status and available disk space
- **Database connection errors:** Verify PostgreSQL credentials and network connectivity
- **OAuth errors:** Confirm redirect URLs match exactly in Google Cloud Console
- **Email delivery failures:** Verify Resend API key and domain verification status

For additional support, consult the respective documentation for Dokploy, Supabase, and SvelteKit.

Database Migration Management >

Comprehensive guide to managing Supabase database migrations for SEMS2